Milestone 6 Written Analysis: Observability and Automated Deployment

Team Mahokshahvata · https://34-8-137-93.sslip.io · repo tharun-yatarla02/Mahokshahvata, branch master · GKE us-central1-a

What the logs and dashboard monitor, and how 5xx errors are counted

The app writes one JSON object per line to stdout, and GKE ships it to Cloud Logging as a structured entry: `severity` becomes the entry's severity and the other keys land in `jsonPayload`. A middleware writes exactly one line per HTTP request with method, path, status, latency and the commit SHA of the running image. If a handler raises, the middleware still logs status 500 before the exception propagates, so crashes are counted, not only returned errors. uvicorn's startup messages and tracebacks use the same JSON formatter. Successful probe calls to /health and /ready are skipped to keep the logs readable; their failures are logged.

The log-based counter metric `app_http_5xx` counts entries matching `resource.type="k8s_container"`, cluster `mahokshahvata`, container `app` and `jsonPayload.status>=500`. Because there is one line per request, every 5xx response adds exactly one. On Nov 8 three requests to the controlled-error endpoint /debug/fail returned 500, produced three ERROR entries with status 500, and the metric read 3 for that minute (0 before and after). The dashboard "Mahokshahvata app" shows this metric next to the load balancer's requests by response class (2xx–5xx), CPU and memory per pod, container restarts, and the alert's state.

What triggers the alert and where it goes

The policy "App is returning HTTP 5xx" fires when the metric's sum over one minute is above zero, so a single 5xx opens an incident. Notifications go to the email channel sidsidsid328@gmail.com, and a "recovered" email follows once a full minute passes with no 5xx. In the test above both emails arrived: the incident opened about four minutes after the errors (normal ingestion delay for log-based metrics) and recovered 53 seconds later. One error is the right trigger for a low-traffic class project. With real traffic we would alert on a 5xx *rate* (for example above 1% for 5 minutes) so a single bad request doesn't page anyone.

What Terraform manages, and the exceptions

Terraform manages 31 resources: the private services range and peering, the Cloud SQL instance, database and user (private IP only, TLS required), the storage bucket, the three secrets, the app's service account and its six least-privilege bindings, the Artifact Registry repository and cleanup policy, the GKE cluster and its autoscaling node pool, the load balancer's static IP, the metric, notification channel, alert policy and dashboard, and the CI identity (deploy service account, Workload Identity pool and provider, three bindings). The resources created earlier with gcloud scripts were adopted with import blocks. A refreshed *terraform plan* against the live project reports "No changes". State lives in a versioned, private GCS bucket, never in git.

Not managed, on purpose: the Kubernetes objects (the pipeline changes the image on every push and the HPA changes replicas, so Terraform would always see drift); the load balancer, NEGs and managed certificate, which GKE's Ingress controller creates from the Ingress manifest; secret *values* and the database password, kept out of state; the state bucket itself; the budget alerts, which live on the billing account; the separate Firebase project; and the retired Milestone 3 VM.

How the pipeline deploys, checks health and rolls back

Every push to master runs the test suite against PostgreSQL. Only if it passes does the deploy job run. It signs in to Google with GitHub's OIDC token through Workload Identity Federation, so no key file exists, and the provider accepts only this repository's master branch. The job builds and pushes the image tagged with the commit SHA, records the version currently serving as the rollback target, then runs *kubectl set image* and waits up to 5 minutes for the rollout. Because the Deployment uses maxUnavailable 0, old pods keep serving until new pods pass readiness (/ready, which includes a database query). After the rollout, the job polls the public HTTPS URL until /health reports the new SHA and /ready returns 200. If either step fails, the job runs *kubectl rollout undo*, waits, and confirms through the URL that the previous SHA is serving with /ready 200. The run still ends red.

Results: commit 16b8f5f went from push to healthy in 2 min 53 s (run 42454929244). A commit that broke /ready was stopped by the tests and never deployed (run 42455180507). A deliberately misconfigured image, 8faceed, listening on the wrong port, never became ready; its rollout timed out after 5 minutes, and the job rolled back and verified 16b8f5f serving again (run 42455266061).